Autonomous Detection and Localization of Emergency Equipment Using SLAM, Frontier Exploration, and Deep Learning

John Toluwanimi Olatubosun
MSc Robotics and Artificial Intelligence
University of Lincoln

***Abstract*—Manually taking the inventory of emergency equipment can be time-consuming and prone to human error. This is very critical in emergency scenarios where these items have to be located quickly and precisely. This study implements a fully autonomous robot for emergency equipment detection and localization in unknown environments. It is deployed on a LIMO robot in ROS2 Humble and evaluated in Gazebo simulation. This system achieves fully autonomous operation with frontier-based exploration using m-explore, running in parallel with SLAM using SLAM Toolbox. YOLOv8 working with the RGB-D camera is used to detect and localize emergency equipment, allowing real-time and multi-class detection with a single model. Once the robot has completed exploring the environment, it revisits each detection to confirm the class and location while removing false positives. The system was evaluated across 8 trials in 2 simulated indoor environments, a maze layout and a four-room layout, having 8 emergency objects each. The results show 92.1% map coverage, 95.3% recall with 100% precision, and an average localization error of 0.12m. The verification phase contributed to these results by eliminating false detections and increasing localization precision. The system's ability to operate without prior maps makes it suitable for deployment in emergency response scenarios, with potential for even broader applications by training the object detection model with additional classes.**

Keywords—ROS2, autonomous exploration, frontier-based exploration, SLAM, object detection, emergency equipment, YOLOv8, mobile robotics,

I. Introduction

Emergency equipments such as fire extinguishers and first aid kits are very important safety assets, which are regularly audited to ensure compliance with safety regulations [12]. Manually taking inventory of these items can be labor-intensive, time-consuming, and is also susceptible to human error. An autonomous robotic system can make this process much faster and reduce errors by allowing for more consistent and repeatable inspections without human fatigue or oversight [13]. For such a system to be usable, it needs to be able to function autonomously in unknown environments without prior maps, and provide accurate 3D localization of detected equipment for the inventory reports. This work implements an autonomous robot system that is able to perform Simultaneous Localization and Mapping (SLAM) in a previously unknown environment using frontier-based navigation, and can also detect and localize fire extinguishers and first aid kits using deep learning for object detection.

Most robotic systems use waypoint-based navigation for autonomous exploration, where humans manually select specific locations for the robots to visit. This approach is heavily dependent on the quality of the chosen waypoints, and quality coverage cannot be guaranteed without exhaustive waypoint placement. Quality waypoints can only be selected by having prior knowledge of the environment, which makes this approach unreliable for dynamic environments where there is no prior knowledge of the world. Coverage path planning approaches, such as lawn mower or spiral patterns, offer systematic coverage, but also depend on prior knowledge of the environment. Frontier-based exploration, on the other hand, allows for truly autonomous exploration in unknown environments. It works by detecting and navigating to unexplored regions at the boundary between known free space and unknown space. This approach does not require prior knowledge or manual waypoint selection, making it suitable for deployment in different environments.

This system also requires an object detection strategy to reliably detect and localize emergency equipment across varying conditions, such as lighting and pose. In the course of this study, Haar Cascades were considered but not implemented because it would require extensive manual feature engineering, and it is known to perform poorly on objects with varying appearances, lighting conditions, or partial obstruction. With Haar Cascades, each object would require a separate cascade trained on thousands of positive and negative examples, making it difficult to scale. Another approach that was considered was feature-based matching, using ROS2's find_object_2d package that relies on SIFT, SURF, or ORB features to match template images. While this approach is effective for specific object instances, it struggles with class-based detection and is less robust to viewpoint and lighting variations. This work went on to implement a deep learning object detector, specifically YOLOv8, which allows for a trainable solution with multi-class support in a single model. YOLOv8 provides performant real-time inference, which is essential for exploration tasks, properly handles object variations, and provides a high level of accuracy while still maintaining computational efficiency.

This work provides: (1) autonomous exploration with complete environment coverage without prior maps or manual waypoints, (2) accurate detection and classification of emergency equipment types using real-time object detection, (3) 3D localization of detected objects in global coordinates, (4) multi-angle verification to eliminate false positives, and (5) quantitative results across multiple trials and environment layouts using standard metrics

II. Related Work

Yamauchi [1] introduced frontier-based exploration, where robots identified the boundaries between explored and unexplored space. This allowed for autonomous coverage of the environment without prior maps. Burgard et al. [2] extended this to multi-robot coordination by assigning frontiers based on the potential information gain and travel cost, which led to reduced exploration time when compared to uncoordinated approaches. This work implemented single-

robot frontier exploration using m-explore [3], which builds on Yamauchi's approach for ROS 2 integration, enabling fully autonomous deployment in unknown environments without manual waypoint selection.

Autonomous exploration relies on accurate localization and mapping to navigate safely and avoid redundant coverage. Graph-based SLAM is a suitable approach for this task because it addresses the accumulated drift present in sequential filtering methods, through pose-graph optimization and loop closure detection. Macenski and Jambrecic [4] developed the slam_toolbox specifically for ROS 2, providing online asynchronous mapping with efficient loop closure for long-duration exploration tasks. This tool offers significantly greater scalability when compared to particle filter approaches, because it is able to correct errors that build up over time, and still maintain real-time performance. This study utilized the slam_toolbox in online asynchronous mode for continuous map building during autonomous exploration.

With a reliable method for localization and mapping established, the system also requires a robust object detection method to identify items within the explored areas. Deep learning is a good choice for this purpose because it allows for an end-to-end trainable architecture, rather than feature-engineered approaches. While two-stage detectors, such as R-CNN, can achieve high accuracy through separate region parsing and classification stages, single-stage approaches, including SSD and YOLO, allow for real-time performance because they unify detection and classification into one process. The YOLO family has evolved over successive versions, achieving increasingly better speed-accuracy trade-offs suitable for robotics. Jocher et al. released YOLOv8 [5] with immediate bounding box prediction and optimized performance [6]. This work made use of the YOLOv8m model, trained on datasets from Roboflow Universe, providing robust multi-class detection.

Hoseini et al. [7] showed that multi-view verification can improve object recognition by selecting the best viewpoints based on the appearance of the item. Chen et al. [8] surveyed active vision techniques, showing that viewpoint planning reduces classification uncertainty through geometric consistency checks. This work implemented a multi-angle verification phase (0°, 120°, 240°), where objects confirmed from multiple viewpoints are retained, while unconfirmed detections are eliminated, ensuring zero false positives without additional training.

Robotic inspection systems typically operate in structured environments with predetermined paths. López-Soriano et al. [9] developed an RFID-based inventory robot in order to compare autonomous mapping to waypoint navigation. The study resulted in similar accuracy between both methods; however, waypoint navigation requires prior knowledge of the environment. Yang et al. [10] proposed BIM-based path planning for underground pipeline inspection using predefined waypoint sequences extracted from building models. Nguyen et al. [11] demonstrated bridge inspection using UAVs following GPS waypoints with sub-0.5m tracking accuracy. These studies prioritize precise positioning over exploration coverage, while this work focuses on combining frontier exploration with YOLO, deep learning detection, for a fully autonomous system for exploration, detection, and classification.

III. System Design

This system is composed of multiple subsystems that are integrated seamlessly for autonomous operation: SLAM constructs a map of the environment while frontier-based exploration enables autonomous navigation to achieve complete coverage of the environment, deep learning object detection in combination with an RGB-D sensor identifies and localizes emergency equipment during exploration, and a multi-angle verification phase removes all false positives after the exploration is completed. This architecture allows the system to be deployed in unknown environments, and is able to achieve high map coverage and robust object detection.

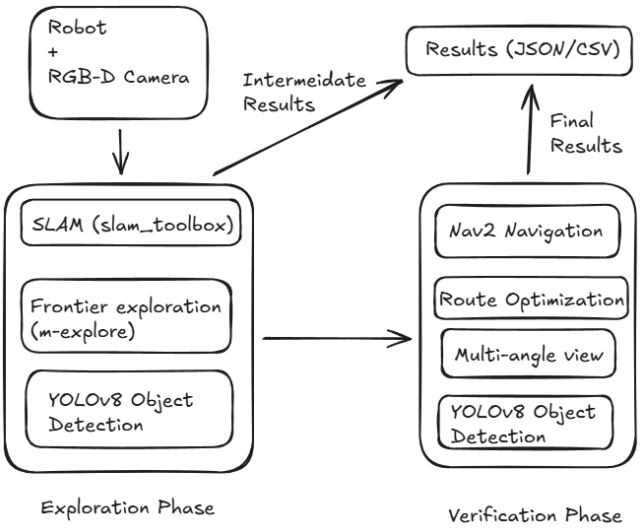

Fig. 1. System architecture showing exploration and verification phases.

A. Object Detection

Object detection requires a robust strategy that can handle variations in appearance, lighting conditions, or partial occlusion. Most traditional computer vision approaches have several limitations: Haar cascades require extensive manual feature engineering and do not perform well with varied object appearances, while feature-based methods like SIFT/SURF (implemented in ROS2's find_object_2d package) only match specific object instances rather than detecting object classes. This work made use of YOLOv8m, which provides an architecture for training on custom datasets, real-time inference and simultaneous multi-class detection in a single network forward pass. The trained model achieved a 95.6% mean Average Precision (mAP) at 0.5 on performance across both fire extinguisher and first aid kit classes. Localization of the detected objects is done by using the depth camera readings, and converting it into the global frame.

As the robot moves around during exploration, it experiences perspective changes and the camera also shakes, which causes the system to sometimes detect the same physical object multiple times but at slightly different locations. To handle this false detections, the system makes use of spatial clustering using Euclidean distance with a 2.0m threshold, to group multiple detections of the same object [14]. The threshold has shown to be able to handle duplicate detections effectively without merging nearby distinct objects. The system picks the detection with the highest confidence score, eliminating false positives.

B. SLAM and Autonomous Exploration

Accurately mapping the environment is essential to both frontier detection and object localization. This work utilizes the slam_toolbox in online asynchronous mode to implement graph-based SLAM. This way, the robot continuously corrects accumulated positioning errors based on new sensor data it receives, allowing it to maintain map accuracy throughout extended exploration. This approach is also very scalable in large environments and allows for real-time map updates during exploration without blocking the robot's motion.

To drive the exploration process for full environment coverage, this project implements frontier-based exploration. Other approaches, like waypoint navigation, require prior knowledge of the environment and are dependent on the quality of the selected waypoints. Coverage path planning methods, such as lawn mower or spiral patterns, also require prior map knowledge and cannot adapt dynamically to different environments. Frontier-based exploration surpasses these limitations by identifying the boundaries between known free spaces and unmapped regions, then setting autonomous navigation goals at those intersections. This project made use of an m-explore fork. The addition to m-explore was durable status publishing using TRANSIENT_LOCAL QoS, which would allow for reliable multi-node coordination.

During the development of this system, a critical relationship was found between the global costmap dimensions and the performance achieved by frontier-based exploration. Frontier detections occurs within the costmap's observable window; so if the costmap is too small, it could cause the exploration process to terminate early, and cause there to be unexplored regions, because the next frontier might be outside the robot's observable window. Increasing the global costmap dimensions allows the robot to properly explore the environment and achieve full coverage. This reveals that costmap sizing must scale with the environment dimensions for a reliable frontier-based exploration. Autonomous navigation, on the other hand, is done with Nav2 by using the DWB local planner for dynamic path planning [15].

C. Verification Phase

A multi-angle verification phase is used to address false positive detections that occur during exploration; these could be due to challenging viewing angles, lighting conditions, or object occlusion. Single-viewpoint verification is unreliable because objects may only be detectable from certain viewing angles, due to pose or lighting conditions. This work implements a verification process that revisits each detection from three viewing angles spaced at 120 degrees (0 degrees, 120 degrees, and 240 degrees), allowing for full coverage. When traveling to each detection, the system employs greedy nearest-neighbor route optimization to minimize the total verification travel distance. Each detection occurs from a standoff position 1.5 meters away, using the global costmap to check whether the viewing position is free; if not, the viewing angle is slightly adjusted to avoid a collision. Objects that cannot be detected from all viewing angles are removed. This method achieved 100% precision across all trials by eliminating all false detections while maintaining high recall.

IV. Results and Discussion

The system was evaluated by testing it in two different simulated worlds, and the results for four trials in each world were recorded. World A has a maze-style layout with narrow corridors, which assesses the robot's ability to effectively navigate through tight spaces with dead ends. World B is made up of four rooms that are connected centrally with a corridor, and each room has partitions; this layout assesses the robot's ability to effectively explore open areas. Each environment contains 8 objects (4 fire extinguishers and 4 first aid kits) with the ground truth positions manually recorded for validation. The robot was initialized from the same starting position for each world across all trials, in order to avoid introducing a bias based on the starting coordinates.

The robots' performance was evaluated using the following metrics: precision, recall, and F1-score. The precision is used to determine the proportion of correct detections among all detections, recall is used to determine the percentage of objects that were successfully detected, and the F1-score represents the harmonic mean of precision and recall. The localization accuracy was measured using average position error, which was calculated by finding the Euclidean distance between the detected position and the ground truth position. Each detection was matched to a ground truth recording by using a 2.5m distance threshold. The efficiency of the system was measured through the exploration time, verification time, and map coverage percentage; to evaluate its feasibility for practical deployment.

Table I below shows the results of the experiments run on the system, aggregated across all trials. From the results, the system achieved 95.3% recall with perfect precision across all 8 trials, and successfully detected 62 out of 64 total objects. World A demonstrated a higher recall (96.9%) compared to World B (93.75%), which can be attributed to the more structured exploration paths in maze-like corridors. The system achieved zero false positives across all trials, validating the multi-angle verification approach. Then, YOLOv8m model achieved consistently high detection confidence across both environments, with an average of 89.8% and minimal variance (±1.3%); this shows that this object detection strategy is robust and reliable. The average position error remained below 0.13 meters, with a low standard deviation of 0.01 meters, demonstrating consistent localization accuracy.

TABLE I. PERFORMANCE RESULTS ACROSS TRIALS

Metric	World A	World B	Overall
Trials	4	4	8
Objects Detected	31/32	30/32	61/64
Recall (%)	96.9 ± 5.4	93.8 ± 10.8	95.3 ± 8.7
Precision (%)	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
F1-Score (%)	98.3 ± 2.9	96.4 ± 6.2	97.4 ± 4.9
Average Confidence (%)	89.0 ± 1.4	90.5 ± 0.7	89.8 ± 1.3
Position Error (m)	0.11 ± 0.01	0.12 ± 0.01	0.12 ± 0.01
Exploration Time (min)	26.0 ± 2.2	23.7 ± 3.6	24.8 ± 3.2
Map Coverage (%)	91.1 ± 3.8	93.0 ± 7.0	92.1 ± 5.7

The results show that the robot was able to directly address the objectives stated in the introduction. The robot's perfect precision validates the multi-angle verification process, as it was able to successfully eliminate false positives. High recall

achieved by the robot implies that the robot had very good coverage without requiring prior environmental knowledge. This is further emphasized by the high map coverage recorded of 92.1%. The average localization error is fairly small, allowing us to conclude that the system meets the localization objective. Overall, the robot performed consistently across both environments, which shows that the system is robust to dynamic environments.

The experiments revealed some important factors to consider when working with frontier-based exploration using m-explore. The min_frontier_size parameter creates a trade-off. If the value is set too low, the robot tries to explore every tiny gap in the map, while this is an effective strategy for complete map coverage, it drastically increases the exploration time. Again, if the value is larger, the exploration process is sped up, but the robot might miss small areas. For this experiment, the min_frontier_size was set to 0.335 meters, which is a good balance; in only 2 out of 8 trials were small sections missed.

The cost-based approach used to select frontiers also has some limitations to consider. When selecting frontiers, the system picks the best one based on distance and size. It does not factor in the current location of the robot, which means that the robot would move around the map a lot. For example, the robot might be at the north of the map, then travel to the south to explore a frontier, and then travel back to the north when it detects a more favorable frontier. Increasing the planner frequency helped mitigate this to a good extent, but not fully, because the robots would have to move fully into the frontier region; however, the robot could still receive a new goal before fully exploring that region.

To tackle both these issues further work is needed on the system. An adaptive threshold or scale could be used together with min_frontier_size; the value could start with a large value then later shrink to allow exploration of smaller areas. Also, frontiers could be grouped into regions so that all the frontiers in one region are explored before moving on to the next region. Finally, testing the system on a physical LIMO robot would reveal how the system performs with real sensor noise and localization drift that does not exist in simulation.

V. Conclusion

This work shows that a system using m-explore based frontier-based exploration for autonomous navigation, and YOLOv8 for object detection, can achieve high performance on emergency items inventory tasks in an unknown environment. It is also important to note that practical deployment would require careful parameter tuning, to balance exploration completeness and efficiency. The multi-angle verification phase perfectly achieved the desired results by eliminating all false positives; this validates its application as a reliable mechanism for high-stakes applications. Overall, the strong performance across structurally diverse environments demonstrates the viability of fully autonomous frontier-based inventory systems for emergency items inventory tasks.

References

[1] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. 1997 IEEE Int. Symp. Computational Intelligence in Robotics and Automation (CIRA)*, Monterey, CA, USA, July 1997, pp. 146–151.

[2] W. Burgard, M. Moors, D. Fox, R. Simmons, and S. Thrun, "Collaborative multi-robot exploration," in *Proc. 2000 IEEE Int. Conf. Robotics and Automation (ICRA)*, San Francisco, CA, USA, Apr. 2000, pp. 476–481.

[3] A. Topiwala, P. Inani, and A. Kathpal, "Frontier based exploration for autonomous robot," arXiv preprint arXiv:1806.03581, 2018.

[4] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *J. Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] V. V. Kukartsev, R. A. Ageev, A. S. Borodulin, A. P. Gantimurov, and I. I. Kleshko, "Deep learning for object detection in images development and evaluation of the YOLOv8 model using Ultralytics and Roboflow libraries," in *Software Engineering Methods Design and Application, CSOC 2024, Lecture Notes in Networks and Systems*, vol. 1118, R. Silhavy and P. Silhavy, Eds. Cham, Switzerland: Springer, 2024, pp. 642–652.

[7] P. Hoseini, S. K. Paul, M. Nicolescu, and M. Nicolescu, "A one-shot next best view system for active object recognition," *Appl. Intell.*, vol. 52, pp. 3126–3145, Aug. 2021.

[8] S. Chen, Y. Li, and N. M. Kwok, "Active vision in robotic systems: A survey of recent developments," *Int. J. Robot. Res.*, vol. 30, no. 11, pp. 1343–1377, Sept. 2011.

[9] S. López-Soriano and R. Pous, "Inventory robots: Performance evaluation of an RFID-based navigation strategy," *IEEE Sensors J.*, vol. 23, no. 13, pp. 14847–14860, July 2023.

[10] Z. Yang, J. Shu, J. Jiang, W. Han, W. Ding, Y. Gao, and Y. Duan, "Automated path-planning strategy for robotic inspection of underground utilities based on building information model," *Comput.-Aided Civ. Infrastruct. Eng.*, Oct. 2025, doi: 10.1111/mice.70107. N. Kitson, A. C. Constantinou, Z. Guo, Y. Liu, and K. Chobtham, "A survey of Bayesian Network structure learning," *Artif. Intell. Rev.*, vol. 56, pp. 8721–8814, 2023.

[11] H. T. Nguyen, S. Garratt, and N. M. Pham, "Intelligent path planning for civil infrastructure inspection with multi-rotor aerial vehicles," *Construction Robotics*, vol. 8, pp. 497–521, Aug. 2024.

[12] Occupational Safety and Health Administration, "Emergency Action Plan," OSHA Standard 29 CFR 1910.38, U.S. Department of Labor, 2024.

[13] G. K. J. Fischer, M. Bergau, D. A. Gómez-Rosal, A. Wachaja, J. Grater, M. Odenweller, U. Piechottka, F. Hoeflinger, N. Gosala, N. Wetzel, D. Buscher, A. Valada, and W. Burgard, "Evaluation of a Smart Mobile Robotic System for Industrial Plant Inspection and Supervision," *IEEE Sensors J.*, vol. 24, no. 13, pp. 19684-19697, July 2024.

[14] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, Portland, OR, USA, Aug. 1996, pp. 226-231.

[15] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, Oct. 2020.